

Introduction to GAMs with GAM.jl

GAM.jl Contributors

Table of contents

Overview	1
Setup	2
Simulating data	2
Fitting a GAM	2
Coefficients and deviance explained	3
Smooth estimates	4
Prediction at new points	5
Smoothing parameters and REML	6
Model diagnostics	7
Summary	7

Overview

A **Generalized Additive Model (GAM)** extends the linear model by replacing linear predictor terms with smooth functions of covariates. The model takes the form:

$$g(\mu_i) = \beta_0 + f_1(x_{1i}) + f_2(x_{2i}) + \cdots + f_p(x_{pi})$$

where g is a link function, $\mu_i = \mathbb{E}(y_i|\mathbf{x}_i)$, and each f_j is a smooth function estimated from the data using penalized regression splines.

The smooth functions are represented as linear combinations of basis functions:

$$f_j(x) = \sum_{k=1}^{K_j} \beta_{jk} b_{jk}(x)$$

Smoothness is controlled by a wiggleness penalty, and the smoothing parameter λ is estimated automatically (e.g., by REML).

This vignette demonstrates fitting a simple GAM to simulated data using **GAM.jl**.

Setup

```
using GAM
using StatsAPI: nobs, deviance, predict, coef, residuals, r2
using DataFrames
using Random
using Plots
```

Simulating data

We simulate $n = 200$ observations from a sine curve with Gaussian noise:

$$y_i = \sin(2\pi x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, 0.3^2)$$

```
Random.seed!(42)
n = 200
x = sort(rand(n))
y = sin.(2 .* x) .+ 0.3 .* randn(n)
df = DataFrame(x = x, y = y)
first(df, 5)
```

	x	y
	Float64	Float64
1	0.000715255	0.233397
2	0.016095	0.312034
3	0.0269848	0.12494
4	0.0369673	0.0640899
5	0.0424826	0.649839

Fitting a GAM

We fit a GAM with a cubic regression spline (`bs=:cr`) smooth of `x` using 15 basis functions:

```
m = gam(@gam_formula(y ~ s(x, k = 15, bs = :cr)), df)
```

Generalized Additive Model

Formula: $y \sim 1$

Family: Normal

Link: IdentityLink

Method: REML

Parametric coefficients:

	Coef.	Std. Error	t	Pr(> t)
(Intercept)	0.016693	0.0225646	0.74	0.4603

Approximate significance of smooth terms:

Smooth	edf	Ref.df
s(x,bs=cr)	7.50	14

R^2 (adj) = 0.830 Deviance explained = 83.7%

Scale est. = 0.1018 n = 200

The model summary shows parametric coefficients, smooth term EDF, deviance explained, and scale estimate.

Coefficients and deviance explained

```
println("Number of observations: ", nobs(m))
println("EDF per smooth:         ", round(edf(m); digits = 2))
println("Deviance explained:     ", round(r2(m) * 100; digits = 1), "%")
println("Scale estimate:         ", round(m.scale; digits = 4))
```

```
Number of observations: 200
EDF per smooth:         [7.5]
Deviance explained:     83.7%
Scale estimate:         0.1018
```

Smooth estimates

The `smooth_estimates()` function evaluates the estimated smooth on a regular grid and returns pointwise standard errors:

```
se = smooth_estimates(m; n = 200)
x_grid = se.covariates[:x]
f_hat = se.estimate
f_se = se.se
```

200-element Vector{Float64}:

```
0.1504749049632283
0.14098547076131987
0.13178252016829053
0.12294428522335005
0.1145521765821661
0.10668966258566856
0.0994400375940059
0.09288260804174672
0.08708690941567825
0.08210494696309274

0.05794663934229006
0.059216301697534765
0.06110082747644667
0.06387627200007191
0.06775730168321518
0.07284430663004396
0.07911056590251758
0.08642398090733551
0.09458428948228123
```

We plot the smooth estimate with ± 2 SE confidence bands, overlaying the true function:

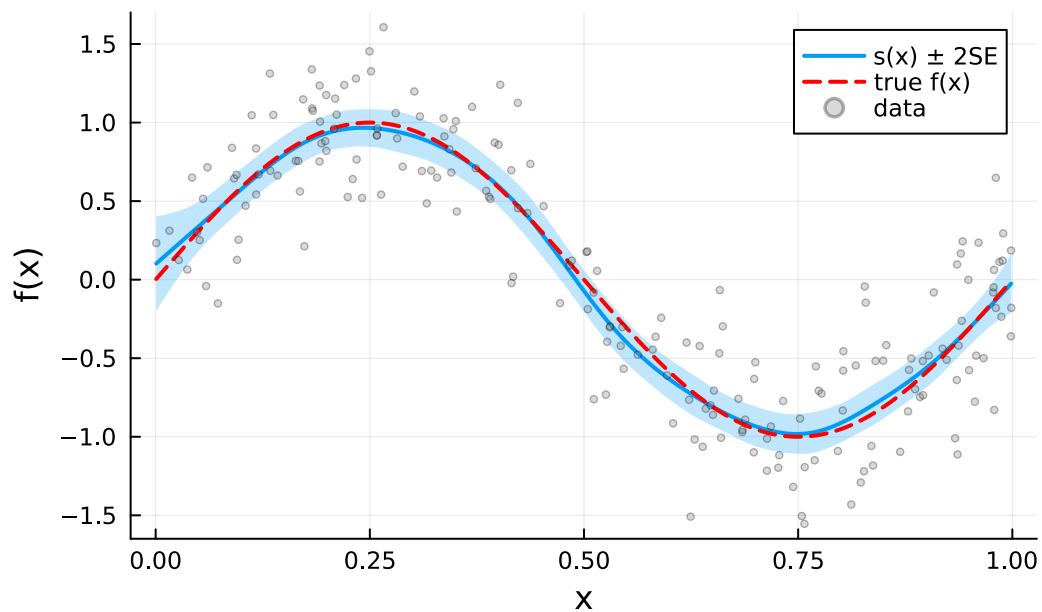
```
plot(x_grid, f_hat;
     ribbon = 2 .* f_se,
     fillalpha = 0.25,
     label = "s(x)  $\pm$  2SE",
     xlabel = "x",
     ylabel = "f(x)",
     title = "Estimated smooth with confidence band",
```

```

    linewidth = 2)
plot!(x_grid, sin.(2 .* x_grid);
    label = "true f(x)",
    linestyle = :dash,
    linewidth = 2,
    color = :red)
scatter!(df.x, df.y;
    label = "data",
    alpha = 0.3,
    markersize = 2,
    color = :gray)

```

Estimated smooth with confidence band



Prediction at new points

We can predict at new covariate values, optionally with standard errors:

```

x_new = DataFrame(x = collect(range(0, 1; length = 50)))
_hat, se_hat = predict(m, x_new; type = :response, se = true)

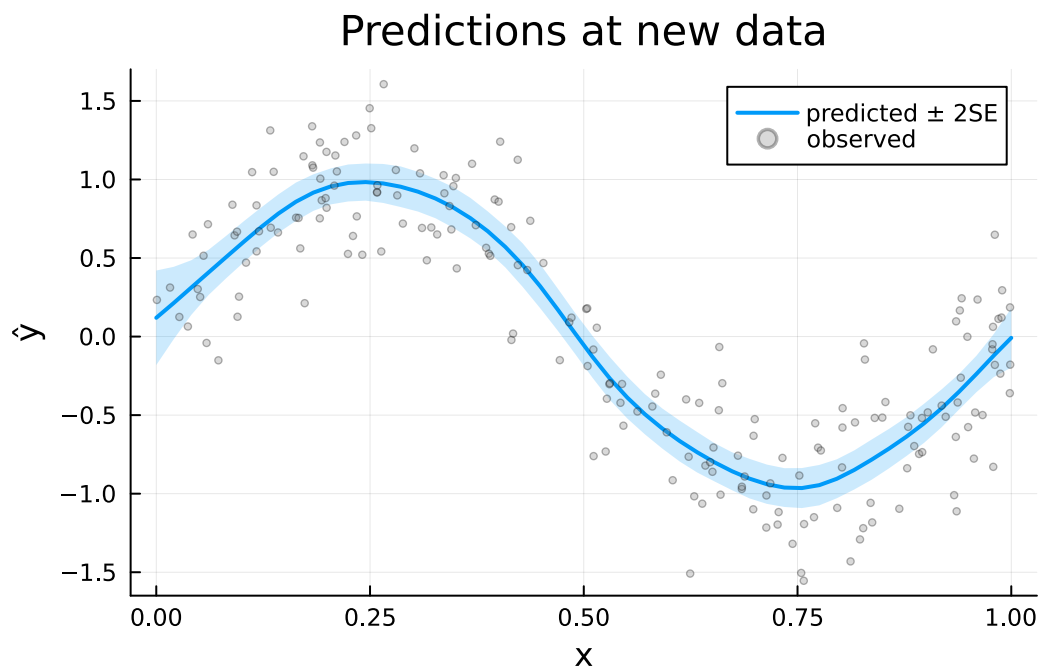
plot(x_new.x, _hat;
    ribbon = 2 .* se_hat,

```

```

fillalpha = 0.2,
label = "predicted  $\pm$  2SE",
xlabel = "x",
ylabel = " $\hat{y}$ ",
title = "Predictions at new data",
linewidth = 2)
scatter!(df.x, df.y;
label = "observed",
alpha = 0.3,
markersize = 2,
color = :gray)

```



Smoothing parameters and REML

GAM.jl estimates the smoothing parameter λ by minimizing the Restricted Maximum Likelihood (REML) criterion by default. REML is generally preferred over GCV because it:

- Avoids the tendency to undersmooth
- Provides more stable estimates
- Has a natural Bayesian interpretation

The fitted model stores the estimated smoothing parameters:

```
println("Estimation method: ", m.method)
```

Estimation method: REML

Model diagnostics

The `gam_check()` function provides a quick diagnostic summary:

```
gam_check(m)
```

GAM checking results

=====

Model converged

Method: REML

Scale est. = 0.1018

n = 200

Basis dimension (k) checking results:

Smooth	k'	edf	k-index
s(x,bs=cr)	14	7.50	0.536

Deviance explained = 83.7%

Scale (σ^2) = 0.1018

Summary

In this vignette we:

1. Simulated data from a sine curve with Gaussian noise
2. Fitted a GAM using a cubic regression spline with $k = 15$ basis functions
3. Examined the model summary, EDF, and deviance explained
4. Visualized the smooth estimate with confidence bands
5. Predicted at new data points with standard errors
6. Discussed REML-based smoothing parameter estimation

The next vignette compares different smooth basis types.